Section 3.4: The Multi-Layer Perceptron

From a Toy Network to a General Architecture

AI-IM Class — Lecture Notes [Student Copy]

In Section 3.3 we computed the forward pass *by hand* and saw how backpropagation trains a tiny network: two inputs, two hidden neurons and one output. Real networks are built from exactly the same pieces, only more of them. In this section we step back and look at the network as a whole:

- how the general architecture is organised and how big it can grow,
- how to predict several outputs at once,
- which activation functions are used in practice,
- how gradient descent is run on large datasets, and
- how to change the output layer and loss for classification.

1 General Architecture

1.1 Layers and Nodes

A **multi-layer perceptron (MLP)**, also called a **fully connected feedforward network**, is a stack of *layers*. Each layer is a column of *nodes* (neurons):

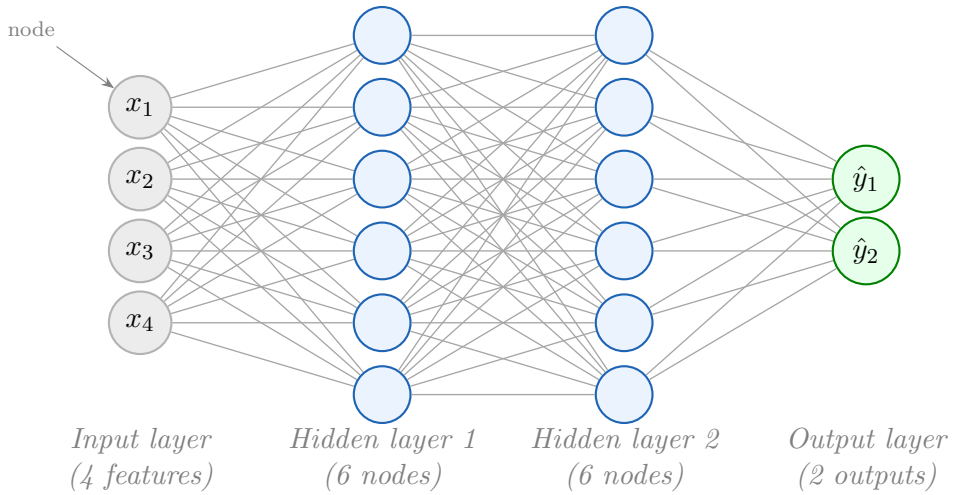


Figure 1: A multi-layer perceptron with four inputs, two hidden layers of six nodes each, and two outputs. Every node is connected to every node in the next layer.

- The **input layer** simply holds the feature values $x_1, \ldots, x_d$. It has no weights and does no computation.
- The **hidden layers** transform the data step by step. They are "hidden" because we never observe their values in the data; the network invents them (recall the learned features h_1, h_2 of XOR in Section 3.1).
- The **output layer** produces the prediction(s) $\hat{y}$.

- The number of nodes in a layer is its **width**; the number of layers with weights is the network's **depth**.

Our XOR network from Section 3.3 is the special case with 2 inputs, one hidden layer of width 2, and 1 output.

1.2 Inside a Node

Every hidden or output node does the same two sub-steps that we saw in Section 3.3: a weighted sum, then an activation.

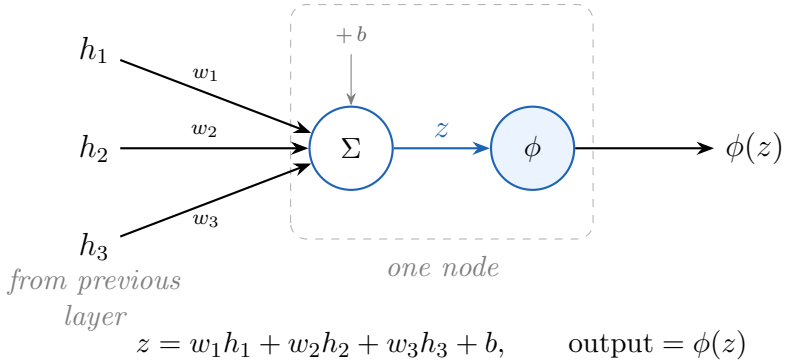


Figure 2: The computation inside a single node. Σ forms the pre-activation z; the activation function ϕ is then applied to it.

We write ϕ for a generic activation function. In Section 3.3 we used $\phi = \sigma$; the *Activation Functions* part below introduces other choices.

The whole layer at once. Stacking the nodes of layer ℓ gives the same matrix form we used in Section 3.3. With $\mathbf{h}^{(0)} = \mathbf{x}$ (the input),

$$\mathbf{z}^{(\ell)} = \mathbf{W}^{(\ell)}\,\mathbf{h}^{(\ell-1)} + \mathbf{b}^{(\ell)}, \qquad \mathbf{h}^{(\ell)} = \phi\big(\mathbf{z}^{(\ell)}\big), \qquad \ell = 1, 2, \ldots, L.$$

If layer $\ell - 1$ has $n_{\ell-1}$ nodes and layer ℓ has n_ℓ nodes, then $\mathbf{W}^{(\ell)}$ is an $n_\ell \times n_{\ell-1}$ matrix (row i holds the weights feeding into node i) and $\mathbf{b}^{(\ell)}$ has n_ℓ entries. The forward pass is just this pair of equations repeated layer after layer.

Regression output. For a regression problem the target y can be any real number, so we do *not* squeeze the final output through a sigmoid. The output layer uses the **identity** ("linear") activation: $\hat{y} = z^{(L)}$.

Counting parameters. Each node in layer ℓ has one weight per incoming node plus one bias, so

$$\#\text{parameters in layer } \ell = \underbrace{n_{\ell-1} \times n_\ell}_{\text{weights}} + \underbrace{n_\ell}_{\text{biases}} = (n_{\ell-1} + 1)\, n_\ell.$$

1.3 More Nodes, More Flexibility

To see what extra nodes and layers buy us, we fit a single-input, single-output regression problem: the blue curve below is a wiggly, noisy function of x, and we fit it with increasingly large networks.

Step 0: No hidden layer. With the input connected straight to a linear output node, the network is

$$\hat{y} = w\, x + b,$$

which is ordinary linear regression with 2 parameters. A straight line cannot follow the bumps.



Figure 3: No hidden layer: the input feeds a linear output node directly.

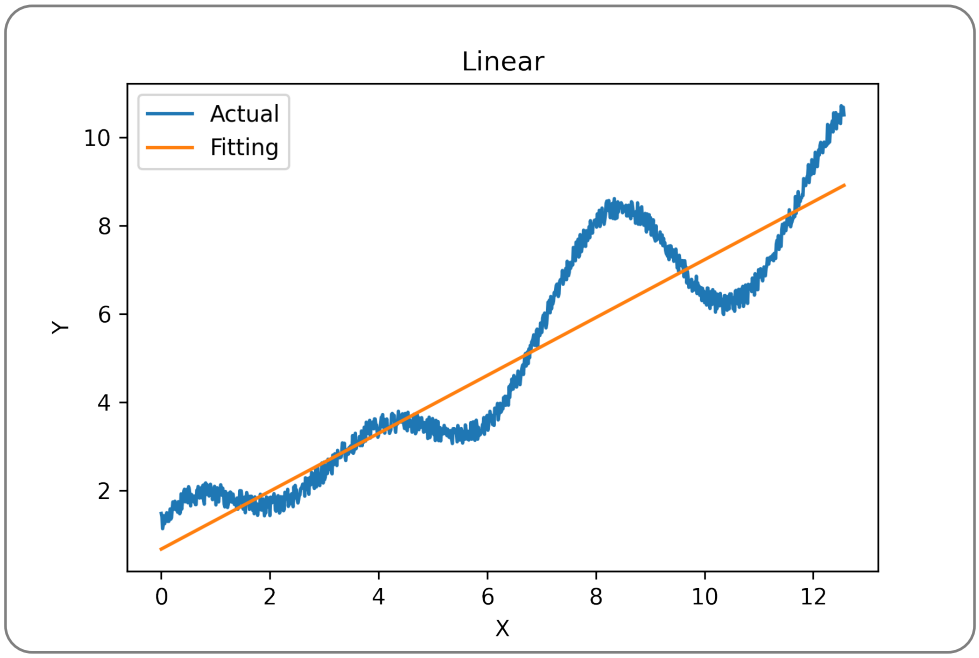


Figure 4: No hidden layer: $\hat{y} = wx + b$ (2 parameters).

Step 1: One hidden node. Insert one sigmoid node between input and output:

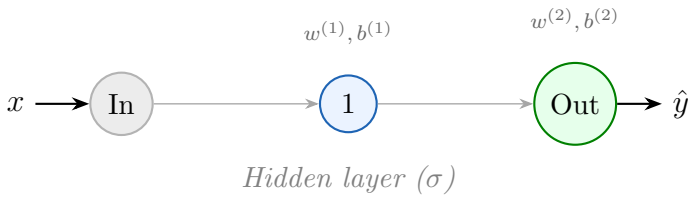


Figure 5: One hidden layer with a single sigmoid node and a linear output.

$$\hat{y} = w^{(2)}\,\sigma\big(w^{(1)}x + b^{(1)}\big) + b^{(2)} = \frac{w^{(2)}}{1 + e^{-(w^{(1)}x + b^{(1)})}} + b^{(2)}.$$

Now the fit is a single "S" that has been shifted and stretched. It follows the overall rise of the data but not the bumps (4 parameters).

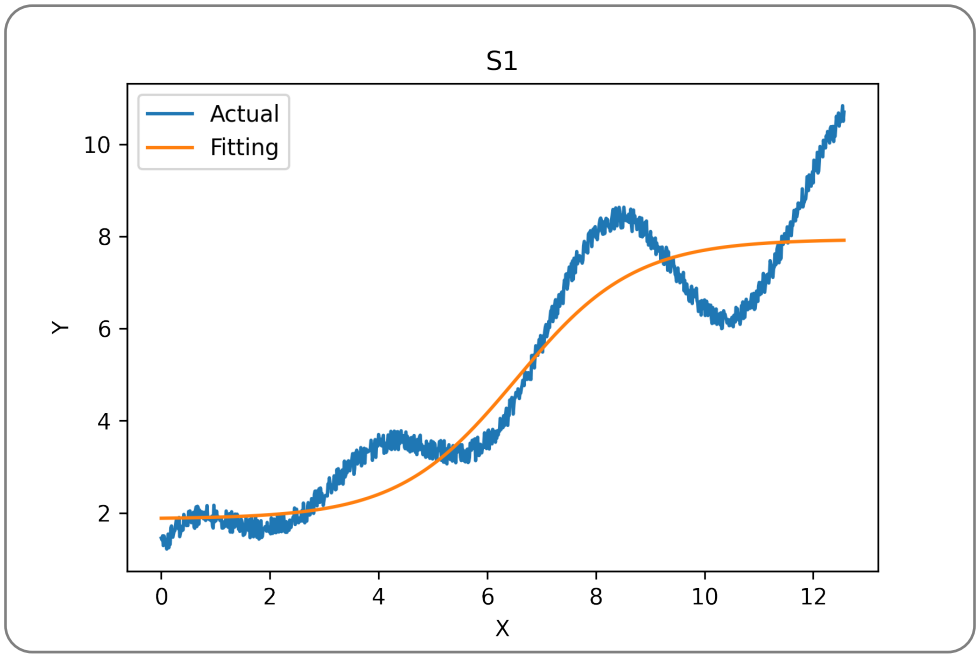


Figure 6: One hidden node with sigmoid activation (4 parameters).

Step 2: Ten hidden nodes. Widen the hidden layer to ten nodes:

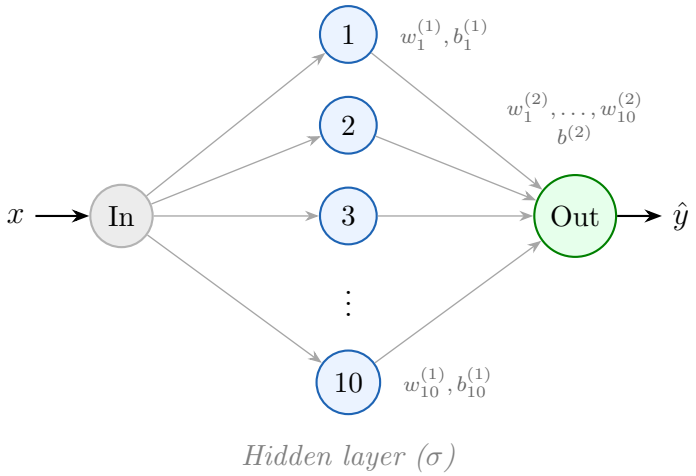


Figure 7: One hidden layer with ten sigmoid nodes and a linear output.

The prediction is a *weighted sum of ten S-curves*:

$$\hat{y} = \sum_{j=1}^{10} w_j^{(2)}\,\sigma\big(w_j^{(1)}x + b_j^{(1)}\big) + b^{(2)} = \sum_{j=1}^{10} \frac{w_j^{(2)}}{1 + e^{-(w_j^{(1)}x + b_j^{(1)})}} + b^{(2)}.$$

Each S-curve can sit at a different place and have a different height and steepness, so together they can trace the bumps of the data. Count the parameters:

$$\#\text{params} = \underbrace{(1+1)\times 10}_{\text{hidden}} + \underbrace{(10+1)\times 1}_{\text{output}} = _____$$

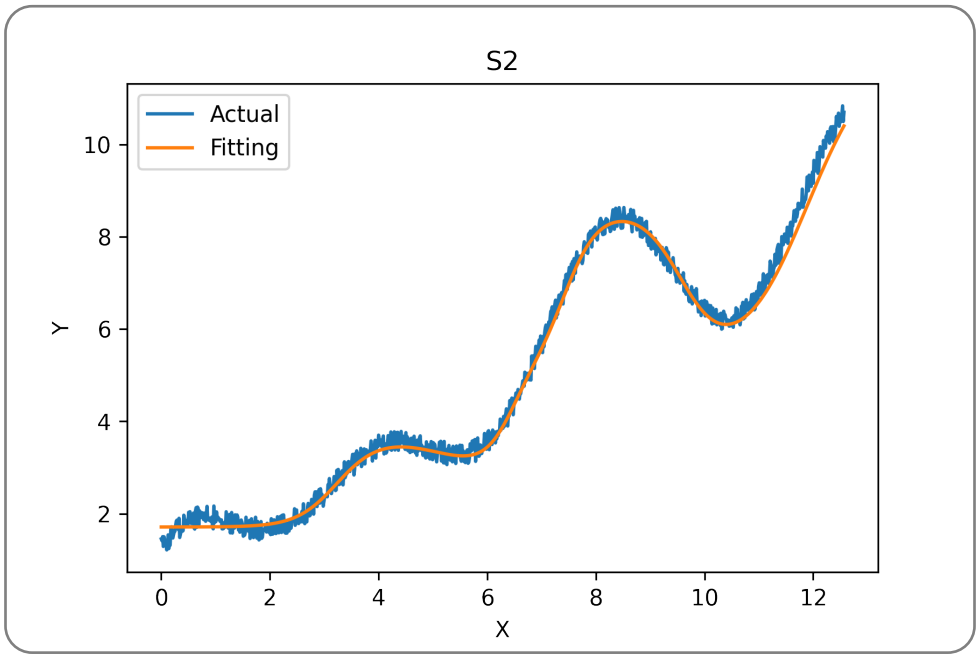


Figure 8: One hidden layer, ten sigmoid nodes. The fit now follows every bump.

Step 3: Two hidden layers of 64 nodes. Now stack two wide layers: $1 \to 64 \to 64 \to 1$.

Layer	$n_{\ell-1}$	n_ℓ	Parameters $(n_{\ell-1}+1)\,n_\ell$
Hidden 1	1	64	_____
Hidden 2	64	64	_____
Output	64	1	_____
Total			_____

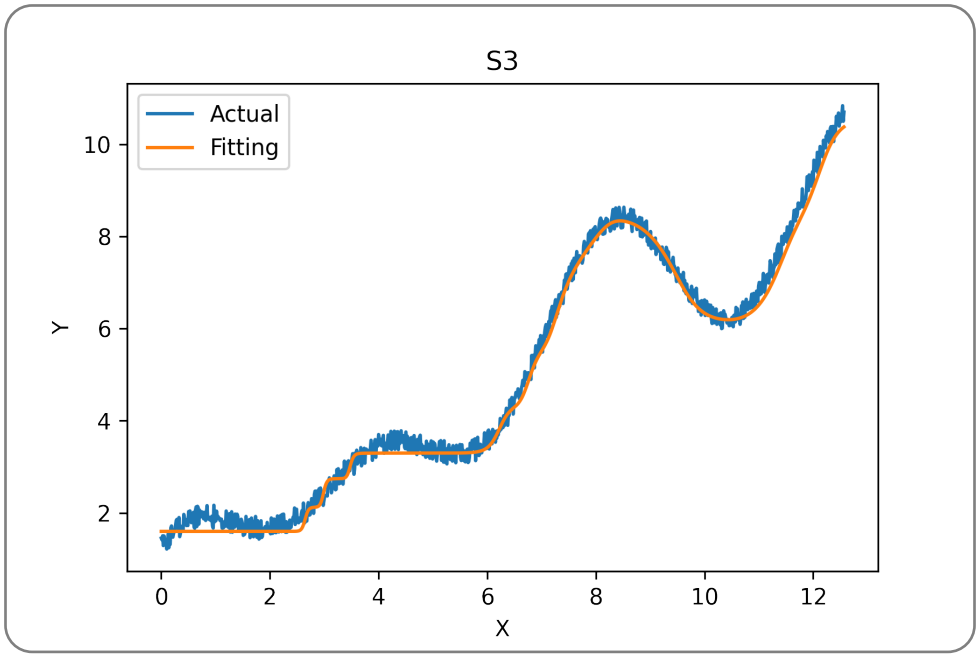


Figure 9: Two hidden layers of 64 nodes each.

1.4 Deep Neural Networks

A network with many hidden layers is called a **deep neural network**; this is where the name *deep learning* comes from. A common design makes the layers wide near the input and gradually narrower towards the output:

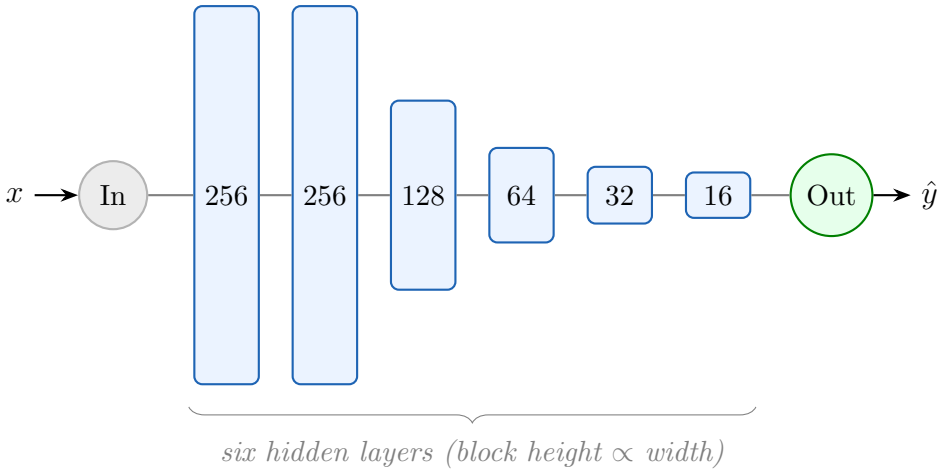


Figure 10: A deep network: $1 \to 256 \to 256 \to 128 \to 64 \to 32 \to 16 \to 1$. Each block is a fully connected layer; the number is its width.

Layer	$n_{\ell-1}$	n_ℓ	Parameters
Hidden 1	1	256	512
Hidden 2	256	256	65 792
Hidden 3	256	128	32 896
Hidden 4	128	64	_____
Hidden 5	64	32	_____
Hidden 6	32	16	_____
Output	16	1	_____
Total			_____

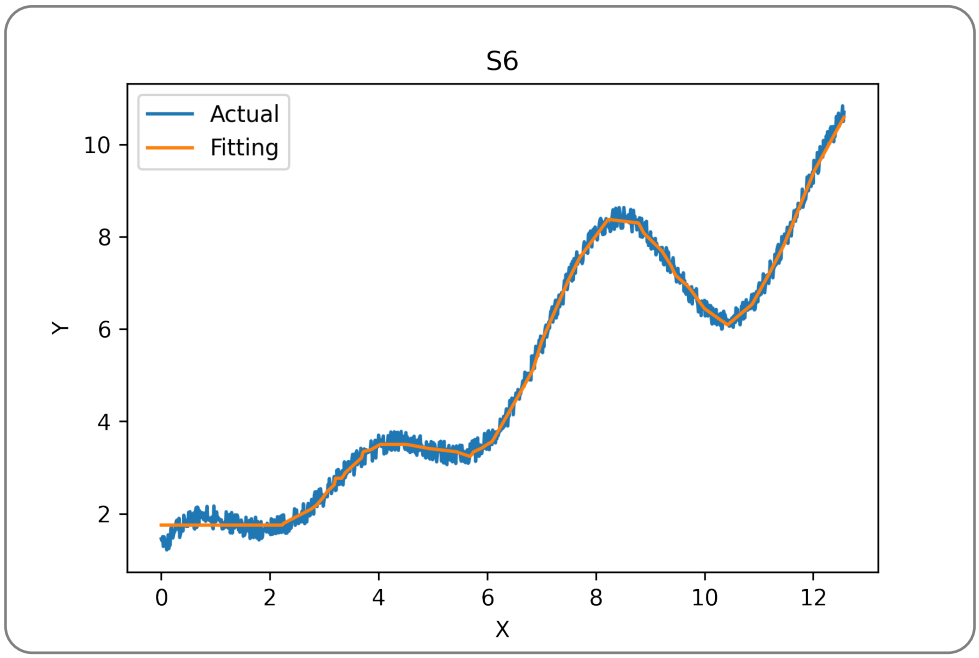


Figure 11: The deep network of Figure 10. The fit is made of short straight segments, the signature of the ReLU activation (see Activation Functions below).

More parameters is not automatically better. The 31-parameter network already fits this data well. The extra parameters of the larger networks mostly go into following the *noise*, and they cost more data and computation to train. We will return to this trade-off (overfitting) later in the course.

Exercise 1 Count the parameters

A network takes 3 input features, has two hidden layers of widths 5 and 4, and produces 2 outputs. How many weights and biases does it have?

#params = _____

2 Multiple Outputs (Regression)

Nothing forces the output layer to have a single node. If we want to predict K quantities at once, for example the demand for three products next week, we simply put K nodes in the output layer, one per target:

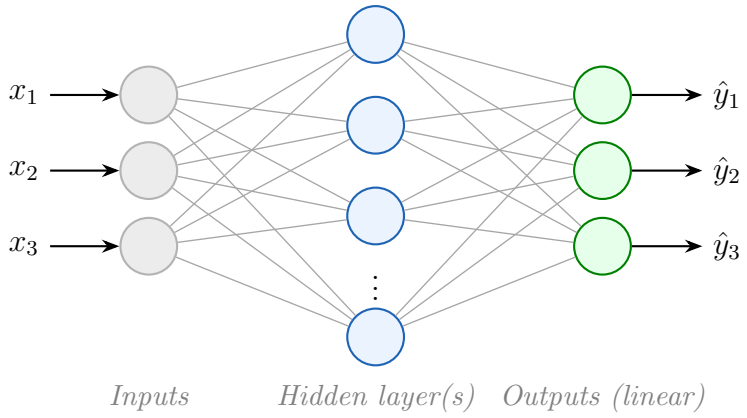


Figure 12: A network with three inputs and three outputs. Each output node is a linear node with its own weights and bias.

The hidden layers are *shared*: all three predictions are built from the same learned features. Only the last layer has separate weights for each output. The loss simply adds up the squared errors over all outputs and all samples:

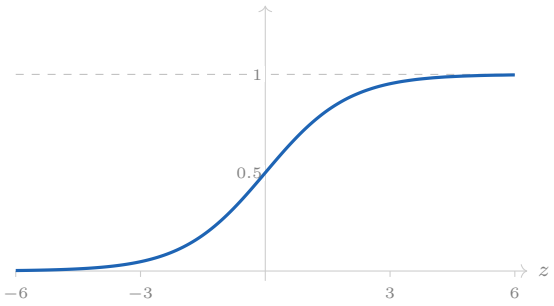
$$J = \frac{1}{n} \sum_{i=1}^{n} \sum_{k=1}^{K} \left(y_k^{(i)} - \hat{y}_k^{(i)} \right)^2.$$

Backpropagation works exactly as before: each output node receives its own error signal $\hat{y}_k - y_k$, and the hidden nodes collect the signals from *all* outputs they feed into.

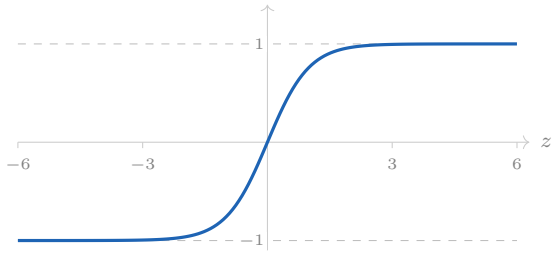
Mind the scales. If one target is measured in thousands and another in fractions, the large one dominates the sum. It is good practice to standardise each target (and each input) before training.

3 Activation Functions

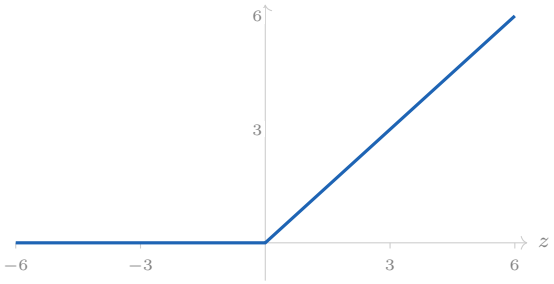
The activation ϕ is what makes a network more than linear regression: without it, stacking layers would just multiply matrices together and the whole network would collapse to a single linear function. Four common choices are shown below.



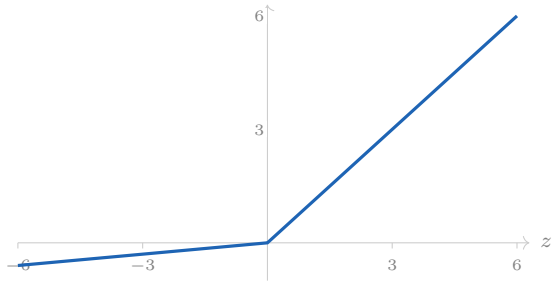
Sigmoid $\sigma(z) = \dfrac{1}{1+e^{-z}}$
range $(0, 1)$



Tanh $\tanh(z) = \dfrac{e^{z}-e^{-z}}{e^{z}+e^{-z}}$
range $(-1, 1)$



ReLU $\mathrm{ReLU}(z) = \max(0, z)$
range $[0, \infty)$



Leaky ReLU $\phi(z) = \max(\alpha z, z)$
range $(-\infty, \infty)$, shown with $\alpha = 0.1$

Figure 13: Four common activation functions.

Why not always use sigmoid? Backpropagation (Section 3.3) multiplies one factor $\phi'(z)$ per layer on the way back. For the sigmoid, $\sigma'(z) = \sigma(z)\big(1-\sigma(z)\big) \le 0.25$, and it is almost 0 whenever $|z|$ is large (the flat parts of the "S"). Through many layers these small factors multiply together, and the gradient reaching the first layers becomes tiny. This is the **vanishing gradient** problem: the early layers barely learn.

ReLU. The **rectified linear unit** has derivative exactly 1 for $z > 0$, so gradients pass through active nodes unchanged. It is also very cheap to compute. ReLU is the default choice for hidden layers in modern networks. Its weakness: for $z < 0$ the derivative is 0, so a node whose pre-activation is always negative stops learning (a "dead" node). **Leaky ReLU** fixes this by giving the negative side a small slope α (typically 0.01–0.1).

Activation	Derivative $\phi'(z)$	Range	Typical use
Sigmoid	$\sigma(z)\big(1-\sigma(z)\big)$	$(0, 1)$	output for binary classification
Tanh	$1-\tanh^2(z)$	$(-1, 1)$	hidden layers (older networks)
ReLU	1 if $z > 0$, else 0	$[0, \infty)$	hidden layers (default)
Leaky ReLU	1 if $z > 0$, else α	$(-\infty, \infty)$	hidden layers
Identity	1	$(-\infty, \infty)$	output for regression

Look back at the fits. With sigmoid hidden nodes (the fits for Steps 1–3 above) the fitted curve is a sum of smooth S-shapes. With ReLU nodes it is a sum of "hinges" $\max(0, wx + b)$, which is why the deep network's fit is made of straight pieces joined at kinks.

4 Optimizer: Gradient Descent on Large Datasets

In Section 3.3 we updated the weights using the gradient from a *single* XOR input. In general the loss is an average over all n training samples,

$$J(\boldsymbol{\theta}) = \frac{1}{n} \sum_{i=1}^{n} J^{(i)}(\boldsymbol{\theta}),$$

where $\boldsymbol{\theta}$ collects every weight and bias. Its gradient is therefore an average of n per-sample gradients. When n is in the millions, computing the exact gradient before *every* small step is far too slow.

Mini-batches. The fix is to **estimate** the gradient from a small random subset of the data, called a **mini-batch** of size B:

$$\nabla J(\boldsymbol{\theta}) \;\approx\; \frac{1}{B} \sum_{i \in \text{batch}} \nabla J^{(i)}(\boldsymbol{\theta}), \qquad \boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \, \nabla J(\boldsymbol{\theta}).$$

The estimate is noisy, but it is cheap, so we can take many more steps in the same amount of time. On average the steps still point downhill.

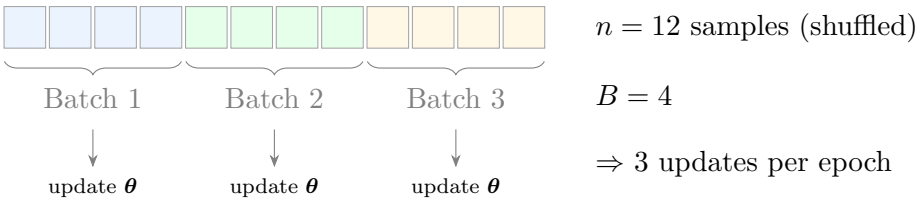


Figure 14: One epoch of mini-batch gradient descent. The shuffled training set is cut into batches; each batch gives one gradient estimate and one update.

One full pass through the training data is called an **epoch**. Training usually runs for many epochs, reshuffling the data each time. Depending on the batch size we get three flavours:

Method	Batch size	Updates / epoch	Character
Batch GD	$B = n$	1	exact gradient, slow per step
Stochastic GD (SGD)	$B = 1$	n	very noisy, cheap per step
Mini-batch GD	$1 < B < n$	$\lceil n/B \rceil$	the practical compromise

In practice "SGD" usually refers to the mini-batch version, with batch sizes such as 32, 64 or 128.

Exercise 2 Updates per epoch

A training set has $n = 10\,000$ samples and we use $B = 64$. How many weight updates happen in one epoch, and in 20 epochs?

Beyond plain SGD. Plain (mini-batch) SGD uses the same learning rate η for every weight. **Adam** (adaptive moment estimation) is SGD with an *adaptive learning rate*: it keeps a running average of past gradients and adjusts the step size for each weight automatically. Adam is the default optimizer in most deep-learning work, and it is the topic of the next section.

5 Classification

So far the outputs were real numbers. For **multiclass classification** the target is one of C categories, for example a product graded A, B or C. Three changes turn the regression network into a classifier: the label encoding, the output layer, and the loss.

5.1 One-Hot Encoding

We cannot feed the labels "A, B, C" into a loss function, and coding them as $1, 2, 3$ would wrongly suggest that C is "more" than A. Instead each label becomes a vector of length C with a 1 in the position of the true class and 0 elsewhere:

Label	Class index	One-hot vector $\mathbf{y}$
A	1	$[1, 0, 0]$
B	2	$[0, 1, 0]$
C	3	$[0, 0, 1]$

The output layer gets **one node per class**, so C output nodes.

5.2 Softmax: From Scores to Probabilities

The output nodes are linear, so they produce arbitrary real numbers $z_1, \ldots, z_C$, called **logits** (raw scores). We want instead a probability for each class. The **softmax** function does this:

$$p_k = \text{softmax}(z_1, \ldots, z_C)_k = \frac{e^{z_k}}{\sum_{j=1}^{C} e^{z_j}}, \qquad k = 1, \ldots, C.$$

Exponentiating makes every score positive, and dividing by the sum makes the scores add up to 1. So $0 < p_k < 1$ and $\sum_k p_k = 1$. The largest logit always gets the largest probability, and the predicted class is the one with the highest p_k.

Exercise 3 Compute a softmax

A network outputs the logits $\mathbf{z} = [2.0,\ 1.0,\ 0.1]$. Compute the class probabilities (to three decimals).

$$e^{2.0} = _____, \quad e^{1.0} = _____, \quad e^{0.1} = _____, \quad \text{sum} = _____$$

$$\mathbf{p} = [_____,\ _____,\ _____]$$

5.3 Categorical Cross-Entropy Loss

In Section 1.2 we derived the binary cross-entropy (BCE) loss for logistic regression. Its multiclass version is the **categorical cross-entropy (CCE)**:

$$J = -\frac{1}{n}\sum_{i=1}^{n}\sum_{k=1}^{C} y_k^{(i)} \log p_k^{(i)}.$$

Because $\mathbf{y}^{(i)}$ is one-hot, only one term of the inner sum is non-zero: the one for the true class. The loss is therefore simply

$$J = -\frac{1}{n}\sum_{i=1}^{n} \log\big(\text{probability the model gives to the } \textit{true} \text{ class of sample } i\big).$$

If the model is confident and right ($p \to 1$), the term $-\log p \to 0$. If it is confident and wrong ($p \to 0$), $-\log p \to \infty$. As in Section 1.2, log is the natural logarithm.

Example 5.1. Two samples with three classes:

$$\begin{aligned} \mathbf{y}^{(1)} &= [0,1,0], \quad \mathbf{p}^{(1)} = [0.05,\ \mathbf{0.95},\ 0.00] \quad \text{(true class 2)} \\ \mathbf{y}^{(2)} &= [0,0,1], \quad \mathbf{p}^{(2)} = [0.10,\ 0.80,\ \mathbf{0.10}] \quad \text{(true class 3)} \end{aligned}$$

Picking out the probability of the true class in each row,

$$J = -\frac{1}{2}\big(\log 0.95 + \log 0.10\big) = -\frac{1}{2}\big(-0.051 - 2.303\big) = 1.177.$$

Almost all of the loss comes from sample 2, which the model gets wrong.

Exercise 4 Loss of a single prediction

For the softmax output of Exercise 3, compute the CCE loss of this one sample if the true class is (a) class 1, (b) class 3.

$$\text{(a)}\ -\log _____ = _____, \qquad \text{(b)}\ -\log _____ = _____$$

5.4 The Complete Classifier

Putting it together, the forward pass of a classifier ends with a softmax and the loss compares its probabilities with the one-hot label:

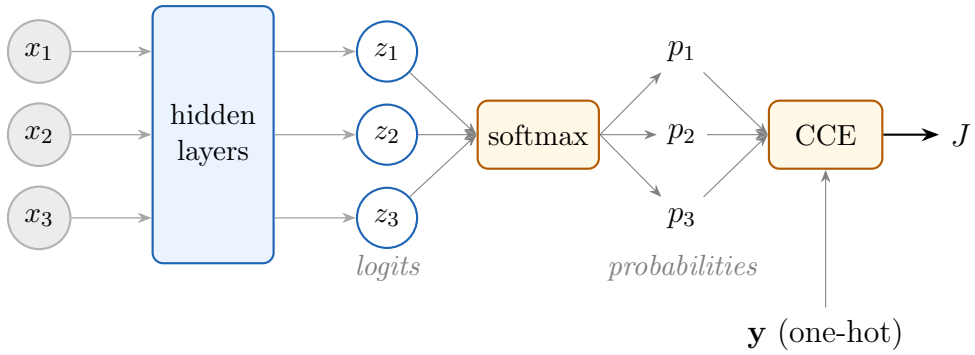


Figure 15: A three-class classifier. The output layer produces logits; softmax turns them into probabilities; the CCE loss compares them with the one-hot label.

A very clean gradient. When softmax is followed by CCE, the error signal at output node k turns out to be simply

$$\frac{\partial J}{\partial z_k} = p_k - y_k,$$

the same "prediction minus target" form we saw for regression in Section 3.3. Back-propagation then continues through the hidden layers exactly as before.

Binary classification. With only two classes we can use a single output node with a sigmoid, $\hat{p} = \sigma(z) =$ probability of class 1, together with the BCE loss from Section 1.2. This is exactly the setup of our XOR network. (A two-node softmax with CCE gives the same model; one sigmoid node is just more economical.)

5.5 Labels and Outputs in Practice

Deep-learning libraries differ in the details, but two choices always have to match what the loss function expects:

1. **How the labels are stored.**
   - As *one-hot vectors*: one row of length C per sample.
   - As an *integer class index*: one number per sample, counting the classes from 0. This saves memory when C is large; the loss picks out p_y directly, which is the same number as $\sum_k y_k \log p_k$.

   Here are three samples ($C = 3$, classes 0, 1, 2) stored both ways:

Sample	True class	One-hot label	Integer label
1	2	$[0, 0, 1]$	2
2	0	$[1, 0, 0]$	0
3	1	$[0, 1, 0]$	1

   For the whole dataset, the one-hot labels form an $n \times C$ matrix (here 3×3), while the integer labels form a vector of length n (here $[2, 0, 1]$). Both carry exactly the same information.

2. **What the network outputs.**

   - *Probabilities*: the model applies softmax itself, e.g. $[0.30, 0.15, 0.55]$. The loss function receives $\mathbf{p}$ and computes only CCE (Figure 16).
   - *Logits*: the model stops at $\mathbf{z}$, e.g. $[1.5, 0.8, 2.1]$. The loss function receives $\mathbf{z}$ and performs softmax *and* CCE together (Figure 17).

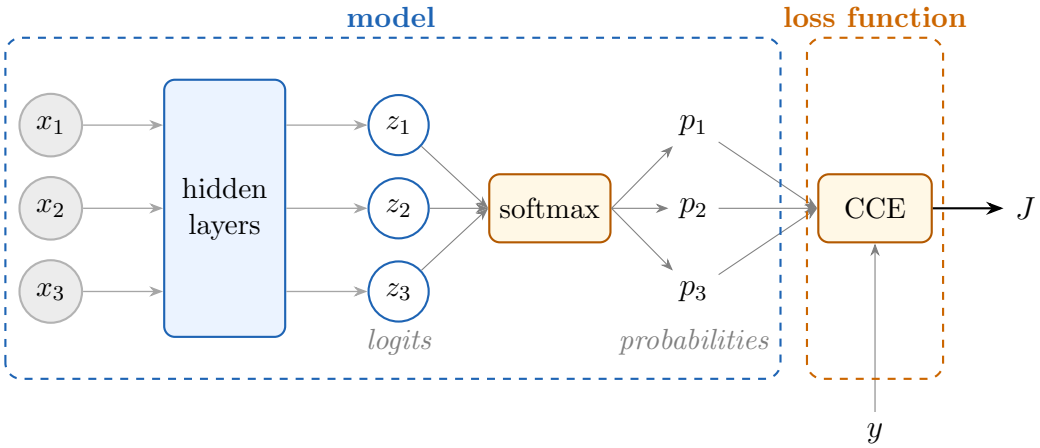


Figure 16: Model outputs *probabilities*. Softmax is part of the model; the loss function receives $\mathbf{p}$ and computes only CCE.

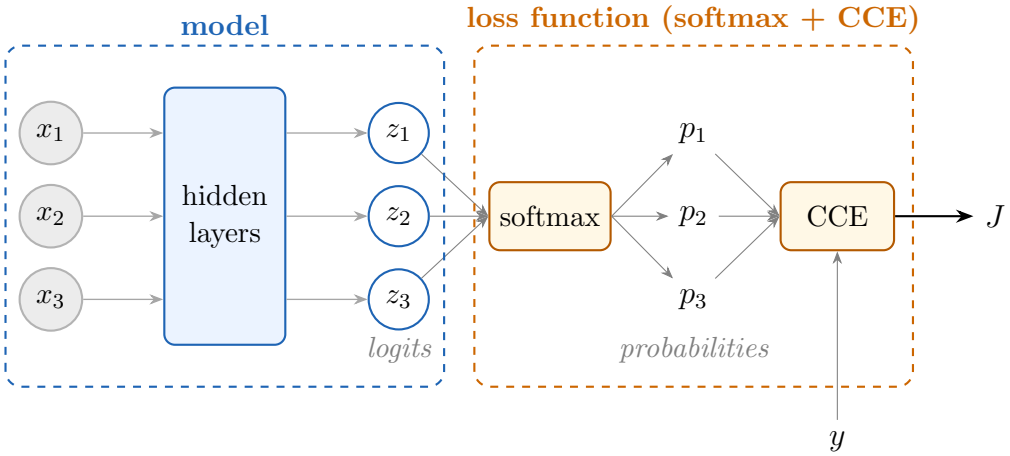


Figure 17: Model outputs *logits*. The model stops at $\mathbf{z}$; the loss function receives the logits and performs softmax and CCE together.

Most libraries prefer logits. Computing softmax and then log separately can overflow (e^z for large z) or take $\log 0$. A loss function that receives the logits can combine both steps in a numerically safe way. The common mistake is to apply softmax in the model *and* again inside the loss; check which form your loss function expects.

6 Summary

Task	Outputs	Output activation	Loss
Regression (one target)	1	identity	MSE
Regression (K targets)	K	identity	MSE (summed)
Binary classification	1	sigmoid	BCE
Multiclass (C classes)	C	softmax	CCE

- An MLP is a stack of fully connected layers. Each layer computes

$$\mathbf{h}^{(\ell)} = \phi\big(\mathbf{W}^{(\ell)}\mathbf{h}^{(\ell-1)} + \mathbf{b}^{(\ell)}\big).$$

- A layer with $n_{\ell-1}$ inputs and n_ℓ nodes has $(n_{\ell-1}+1)\,n_\ell$ parameters. Width and depth add flexibility, but also cost and the risk of fitting noise.
- Hidden layers usually use ReLU (or a variant); sigmoid and tanh suffer from vanishing gradients in deep networks.
- Training uses mini-batch gradient descent: one update per batch, many epochs. Adam (next section) adapts the step size per weight.
- For classification: one-hot labels, C output nodes, softmax, and categorical cross-entropy.